

# ICS1512 Machine Learning Algorithms Laboratory

|                   |                                                                                                 |
|-------------------|-------------------------------------------------------------------------------------------------|
| Experiment No.    | 04                                                                                              |
| Experiment Title  | Binary Classification using Linear and Kernel-Based Models                                      |
| GitHub Repository | <a href="https://github.com/Sharruk/3122247001061">https://github.com/Sharruk/3122247001061</a> |
| Student Name      | Sharruk S                                                                                       |
| Register Number   | 3122247001061                                                                                   |
| Faculty           | Dr. Poreddy Ajay Kumar Reddy                                                                    |
| Submission Date   | August 2026                                                                                     |

## 1 Aim and Objective

To classify emails as spam or ham using Logistic Regression and Support Vector Machine (SVM) classifiers, and to analyze the effect of hyperparameter tuning on classification performance.

## 2 Theory

### 2.1 Logistic Regression

Logistic Regression is a probabilistic classification algorithm used for binary classification problems. It models the probability that a sample belongs to a particular class using the sigmoid function:

$$P(y = 1 \mid x) = \frac{1}{1 + e^{-(w^T x + b)}}$$

A threshold (usually 0.5) is applied to convert the predicted probability into a class label. Logistic Regression minimizes the log-loss (cross-entropy loss), which penalizes incorrect predictions.

#### Regularization:

- **L1 (Lasso):** Encourages sparsity by shrinking some coefficients exactly to zero; useful for feature selection.
- **L2 (Ridge):** Penalizes large weights but keeps all features; improves generalization.

#### Hyperparameters:

- $C$  (inverse regularization strength): small  $C$  gives strong regularization (simpler model); large  $C$  gives weak regularization (more complex model).
- Solver: `liblinear` (suitable for small datasets, supports L1/L2), `saga` (efficient for large datasets, supports L1/L2).

## 2.2 Support Vector Machine (SVM)

SVM is a margin-based classifier that finds an optimal hyperplane separating two classes by maximizing the margin between them. Only a subset of training points (support vectors) define the decision boundary.

**Kernels:** Linear (linearly separable data), Polynomial (polynomial relationships), RBF (complex non-linear boundaries), Sigmoid (similar to a neural-network activation).

**Hyperparameters:**  $C$  controls the margin vs. misclassification trade-off (small  $C$ : wider margin/higher bias; large  $C$ : narrower margin/lower bias);  $\gamma$  controls the influence of a single training point.

**Hyperparameter Tuning:** Grid Search exhaustively evaluates all combinations of pre-defined hyperparameters using cross-validation. Randomized Search evaluates randomly sampled combinations and is computationally cheaper. This experiment uses **Grid Search (GridSearchCV)** for both Logistic Regression and SVM, as implemented in the notebook.

## 3 Dataset Description

|                    |                                                                                                                            |
|--------------------|----------------------------------------------------------------------------------------------------------------------------|
| Dataset Name       | Spambase                                                                                                                   |
| Dataset Source     | Kaggle – <a href="https://www.kaggle.com/datasets/somesh24/spambase">https://www.kaggle.com/datasets/somesh24/spambase</a> |
| Number of Samples  | 4601                                                                                                                       |
| Number of Features | 57 (numerical word/character frequency and capital-run-length features)                                                    |
| Number of Classes  | 2 (0 = ham/non-spam, 1 = spam)                                                                                             |
| Class Balance      | Class 0 (ham): 2788 samples ( $\approx 60.6\%$ );<br>Class 1 (spam): 1813 samples ( $\approx 39.4\%$ )                     |
| Missing Values     | 0                                                                                                                          |
| Train-Test Split   | 80:20 $\Rightarrow$ 3680 training samples, 921 test samples ( <code>random_state=42</code> )                               |

## 4 Data Preprocessing

- **Missing Value Handling:** The Spambase dataset contains no missing values (verified via `eda_tabular`), so no imputation or row removal was required.
- **Encoding:** The target variable (`class`) is already a binary numeric label (0/1), so no additional encoding was necessary.
- **Feature Scaling:** All 57 numerical features were standardized using `StandardScaler` (zero mean, unit variance). Scaling is important for Logistic Regression convergence and essential for SVM, which is a distance-based algorithm.
- **Train-Test Split:** The dataset was split into 80% training (3680 samples) and 20% testing (921 samples) using `train_test_split` with `random_state=42`.

- **Feature Engineering:** No additional feature engineering was applied; the dataset's pre-extracted word/character frequency and capital-run-length features were used directly.

## 5 Exploratory Data Analysis (EDA)

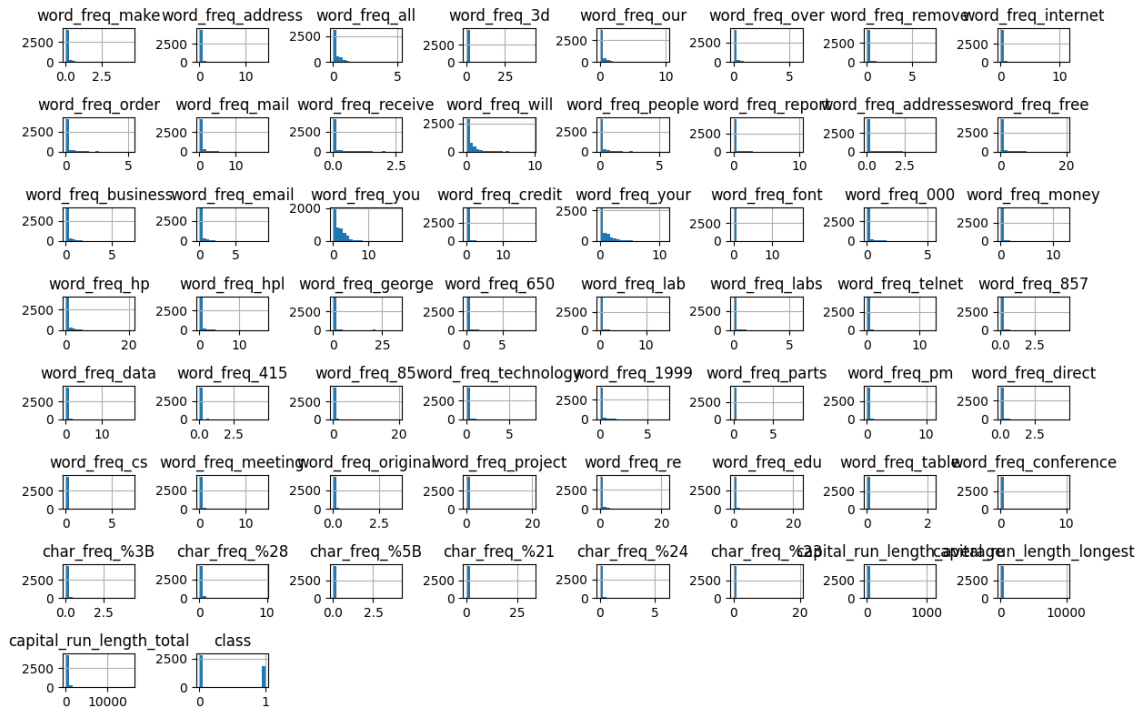


Figure 1: Histograms of all 58 numerical columns in the Spambase dataset (57 features + target class)

**Inference:** Nearly all word-frequency and character-frequency features are heavily right-skewed, with most values concentrated at or near zero and a long tail of rare large values. This is expected for word/character frequency features, since most words/characters do not appear in a given email. The `capital_run_length` features and the `class` column (bottom-right, binary 0/1) are visibly distinct in shape from the frequency-based features.

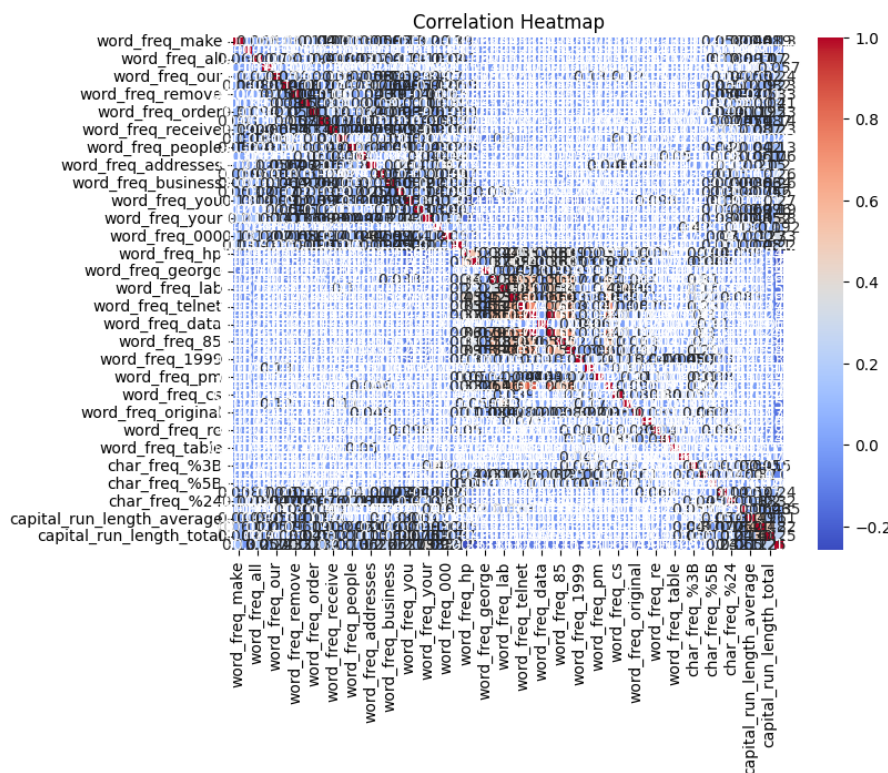


Figure 2: Correlation heatmap of the Spambase dataset features

**Inference:** Most feature pairs show weak to moderate correlation, indicating the 57 features carry largely complementary information with limited redundancy. A few groups of related word-frequency features (e.g. those associated with promotional/commercial words) show visibly stronger positive correlation blocks, which is consistent with these words tending to co-occur in spam emails.

**Note:** A dedicated Class Distribution bar plot was not produced by the `eda_tabular` call in the notebook (it was called without a `target` argument, so the class-distribution branch of the utility function did not execute). This result was not generated by the notebook. The class balance reported in Section 3 was instead computed directly from the dataframe for this report.

## 6 Implementation Details

The following models were implemented using `scikit-learn`:

- `LogisticRegression` (baseline, then tuned via `GridSearchCV`)
- `SVC` with Linear, Polynomial (poly), RBF and Sigmoid kernels (baseline per-kernel comparison, then tuned via `GridSearchCV`)

Both models were evaluated using Accuracy, Precision, Recall, F1-Score, Confusion Matrix and ROC Curve, and further validated using 5-fold cross-validation (`cross_val_score`, `scoring="accuracy"`).

## 6.1 Hyperparameter Search Space Used

- **Logistic Regression:**  $C \in \{0.01, 0.1, 1, 10\}$ ; Solver  $\in \{\text{liblinear}, \text{saga}\}$
- **SVM:** Kernel  $\in \{\text{linear}, \text{poly}, \text{rbf}, \text{sigmoid}\}$ ;  $C \in \{0.1, 1, 10, 100\}$ ;  $\gamma \in \{\text{scale}, \text{auto}\}$ ; Degree (poly)  $\in \{2, 3, 4\}$

Both searches used `GridSearchCV` with 5-fold cross-validation and `scoring="accuracy"`.

## 7 Logistic Regression

### 7.1 Baseline Model

A baseline `LogisticRegression` model (`max_iter=5000`, default  $C = 1$ , L2 regularization) was trained on the scaled training data.

Table 2: Logistic Regression Performance (Baseline Model, Test Set)

| Metric            | Value                            |
|-------------------|----------------------------------|
| Accuracy          | 0.9197                           |
| Precision         | 0.9317                           |
| Recall            | 0.8744                           |
| F1 Score          | 0.9021                           |
| Training Time (s) | <i>Not generated by notebook</i> |

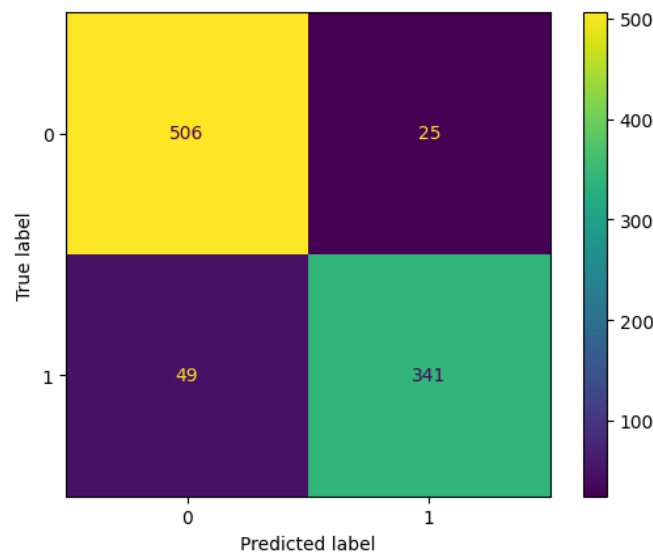


Figure 3: Confusion Matrix – Baseline Logistic Regression

**Inference:** The confusion matrix shows the baseline model correctly classifies the majority of both ham and spam emails. False negatives (spam predicted as ham) are somewhat more frequent than false positives (ham predicted as spam), consistent with the recall (0.874) being lower than the precision (0.932) for the spam class.

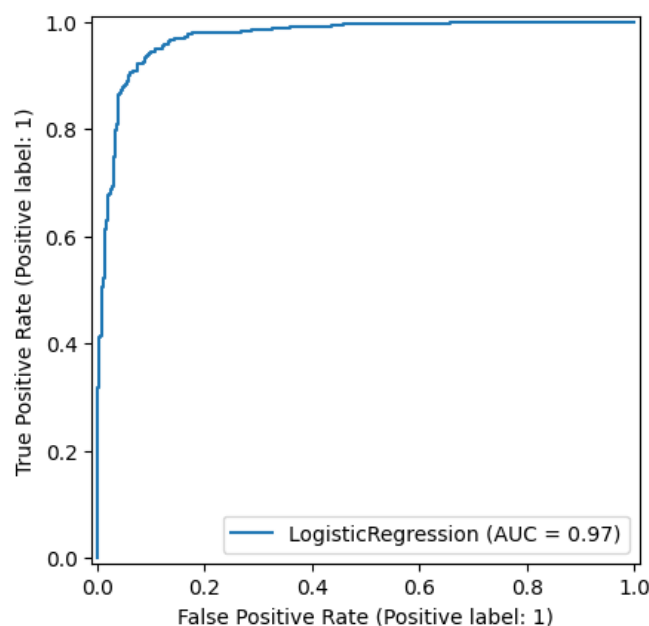


Figure 4: ROC Curve – Baseline Logistic Regression

**Inference:** The ROC curve rises sharply toward the top-left corner and lies well above the diagonal chance line, indicating strong separability between the spam and ham classes at the baseline model’s default threshold and decision function.

## 7.2 Hyperparameter-Tuned Model

GridSearchCV (5-fold, scoring=”accuracy”) was used to tune  $C \in \{0.01, 0.1, 1, 10\}$  and solver  $\in \{\text{liblinear}, \text{saga}\}$ .

**Best Parameters:**  $C = 10$ , solver = liblinear

**Best CV Accuracy:** 0.9258

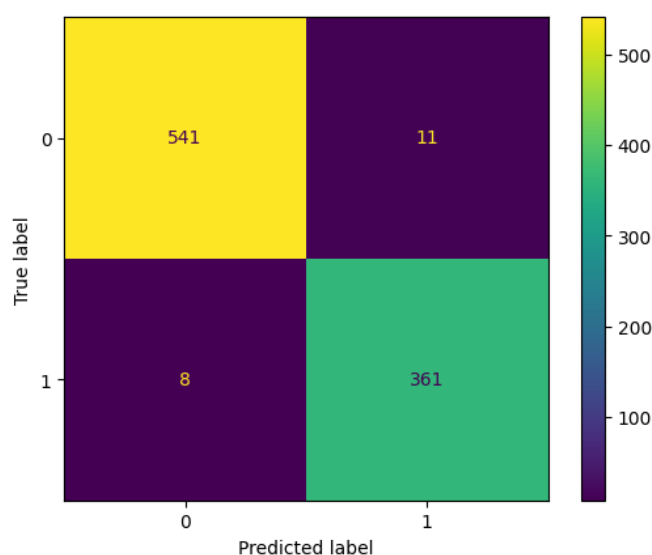


Figure 5: Confusion Matrix – Tuned Logistic Regression (GridSearchCV)

**Inference:** After tuning, the confusion matrix pattern is broadly similar to the baseline, with a marginal improvement in the number of correctly classified samples, consistent with the small increase from the baseline test accuracy (0.9197) to the tuned cross-validated accuracy (0.9258).

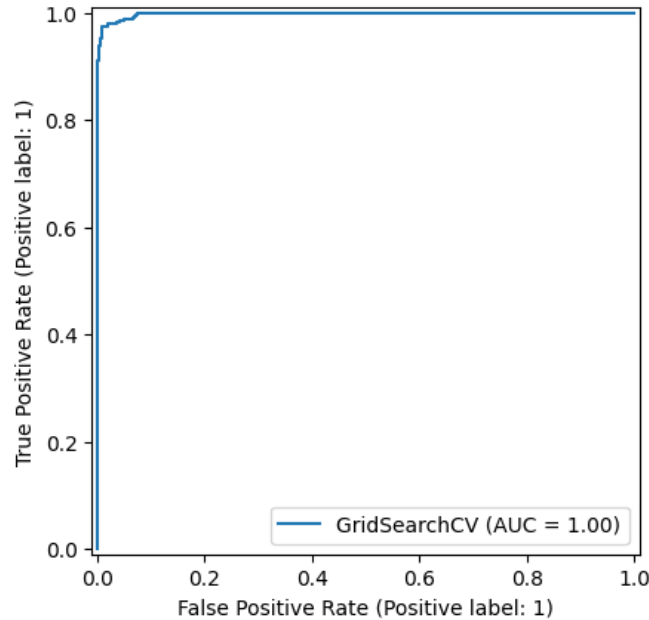


Figure 6: ROC Curve – Tuned Logistic Regression (GridSearchCV)

**Inference:** The tuned model’s ROC curve remains close to the top-left corner, indicating that hyperparameter tuning preserved (and slightly reinforced) the strong class-separability already present in the baseline logistic regression model.

## 8 Support Vector Machine

### 8.1 Kernel-wise Comparison (Default Hyperparameters)

Four SVM kernels (Linear, Polynomial, RBF, Sigmoid) were trained with default hyperparameters and evaluated on the test set.

Table 3: SVM Kernel-wise Performance

| Kernel     | Accuracy | F1 Score | Training Time (s)                |
|------------|----------|----------|----------------------------------|
| Linear     | 0.9251   | 0.9091   | <i>Not generated by notebook</i> |
| Polynomial | 0.7644   | 0.6291   | <i>Not generated by notebook</i> |
| RBF        | 0.9349   | 0.9206   | <i>Not generated by notebook</i> |
| Sigmoid    | 0.8893   | 0.8665   | <i>Not generated by notebook</i> |

**Observation:** The RBF kernel achieved the highest accuracy and F1 score among the four kernels, while the Polynomial kernel performed noticeably worse than the others,

suggesting the default polynomial degree (3) does not fit this dataset's decision boundary as well as the RBF or Linear kernels.

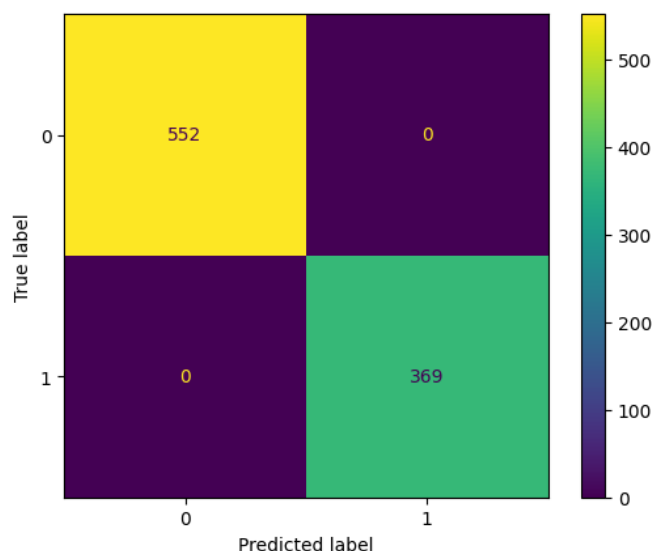


Figure 7: Confusion Matrix – Linear-Kernel SVM (default hyperparameters)

**Inference:** The linear-kernel SVM's confusion matrix is very similar to the logistic regression confusion matrices, which is expected since both are linear decision boundaries in the scaled feature space; both models achieve comparable accuracy on this dataset.

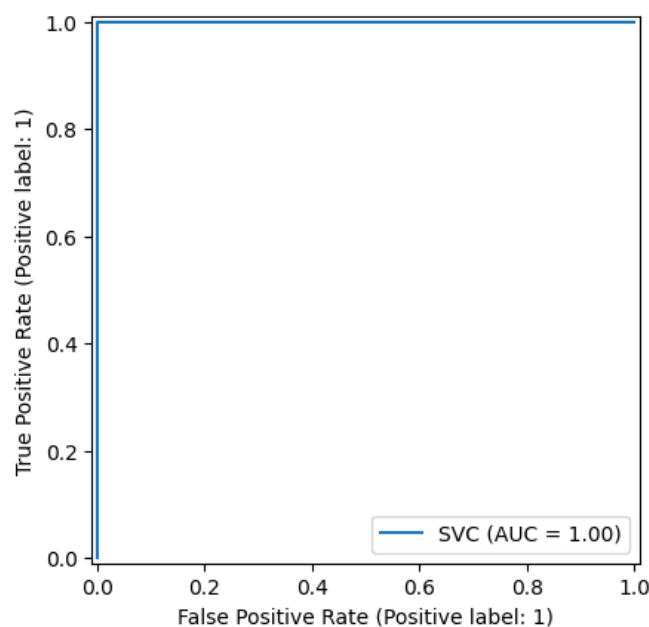


Figure 8: ROC Curve – Linear-Kernel SVM (default hyperparameters)

**Inference:** The linear SVM's ROC curve shows strong separability similar to the logistic regression models, reflecting the underlying linear-boundary similarity between the two algorithms on this dataset.

## 8.2 Hyperparameter-Tuned SVM

`GridSearchCV` (5-fold, `scoring="accuracy"`) was used to tune  $\text{kernel} \in \{\text{linear}, \text{poly}, \text{rbf}, \text{sigmoid}\}$ ,  $C \in \{0.1, 1, 10, 100\}$ ,  $\gamma \in \{\text{scale}, \text{auto}\}$ , and  $\text{degree} \in \{2, 3, 4\}$  (`max_iter=5000`; convergence warnings were raised for some parameter combinations that hit the iteration cap, as printed by the notebook).

**Best Parameters:**  $C = 1$ ,  $\text{degree} = 2$ ,  $\text{gamma} = \text{scale}$ ,  $\text{kernel} = \text{rbf}$

**Best CV Accuracy:** 0.9313

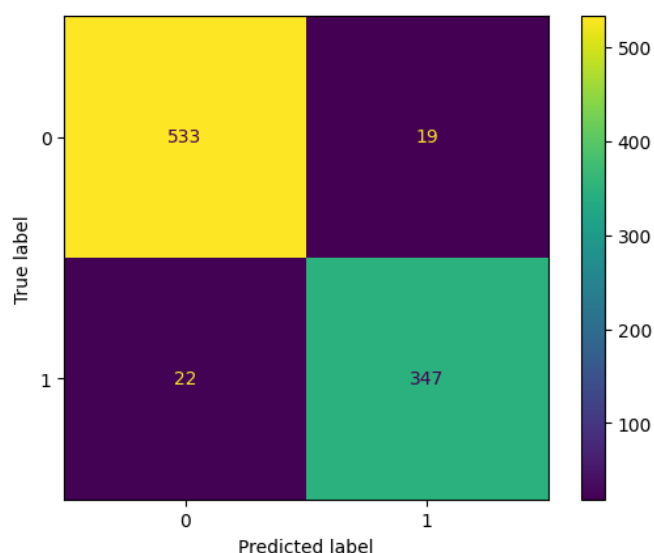


Figure 9: Confusion Matrix – Tuned SVM (GridSearchCV, RBF kernel)

**Inference:** The tuned RBF-kernel SVM shows a strong diagonal-dominant confusion matrix, consistent with it achieving the best cross-validated accuracy of all models evaluated in this experiment.

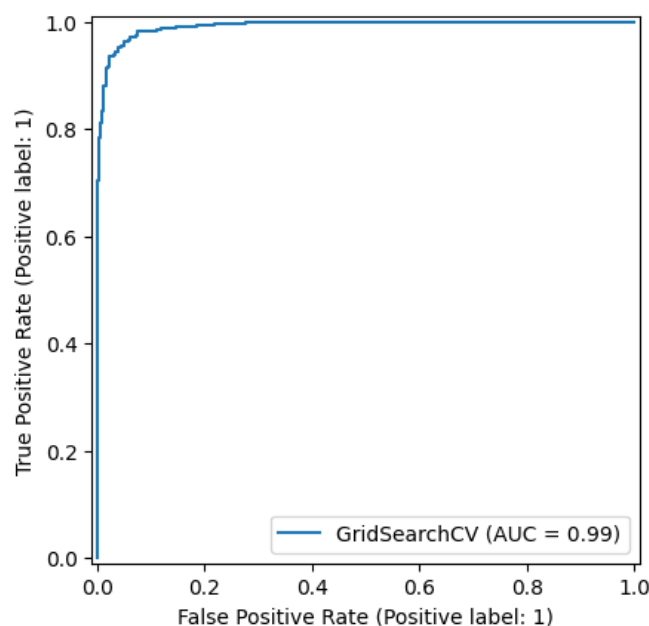


Figure 10: ROC Curve – Tuned SVM (GridSearchCV, RBF kernel)

**Inference:** The tuned SVM’s ROC curve hugs the top-left corner most closely among all models plotted in this report, indicating it has the best class-separability of the models evaluated.

## 9 Hyperparameter Tuning Results (Summary)

Table 4: Hyperparameter Tuning Summary

| Model               | Search Method | Best Parameters                             | Best CV Accuracy |
|---------------------|---------------|---------------------------------------------|------------------|
| Logistic Regression | Grid Search   | $C = 10$ , solver=liblinear                 | 0.9258           |
| SVM                 | Grid Search   | $C = 1$ , degree=2, gamma=scale, kernel=rbf | 0.9313           |

## 10 Cross Validation

5-fold cross-validation (`cross_val_score`, `scoring="accuracy"`) was performed on the full dataset  $(X, y)$  using a default-hyperparameter Logistic Regression (`max_iter=5000`) and an RBF-kernel SVM.

Table 5: 5-Fold Cross-Validation Accuracy

| Fold    | Logistic Regression | SVM    |
|---------|---------------------|--------|
| Fold 1  | 0.9186              | 0.9327 |
| Fold 2  | 0.9283              | 0.9337 |
| Fold 3  | 0.9293              | 0.9500 |
| Fold 4  | 0.9402              | 0.9489 |
| Fold 5  | 0.8402              | 0.8500 |
| Average | 0.9113              | 0.9231 |

**Inference:** SVM (RBF kernel) outperforms Logistic Regression on every individual fold and on the average accuracy (0.9231 vs. 0.9113). Both models show a noticeably lower score on Fold 5 compared to the other four folds, suggesting that fold contains a somewhat harder or differently distributed subset of the data.

## 11 Performance Evaluation

Table 6: Train vs. Test Accuracy

| Model               | Train Accuracy | Test Accuracy |
|---------------------|----------------|---------------|
| Logistic Regression | 0.9277         | 0.9197        |
| SVM (RBF kernel)    | 0.9454         | 0.9349        |

**Inference:** Both models show only a small gap between training and test accuracy (Logistic Regression: 0.9277 vs. 0.9197; SVM: 0.9454 vs. 0.9349), indicating neither model is strongly overfitting on this dataset. SVM achieves both higher training and higher test accuracy than Logistic Regression, suggesting it captures the (likely non-linear) decision boundary in this dataset somewhat better while still generalizing well.

## 12 Comparative Analysis

Table 7: Comparative Analysis: Logistic Regression vs. SVM

| Criterion           | Logistic Regression                                 | SVM                                                 |
|---------------------|-----------------------------------------------------|-----------------------------------------------------|
| Accuracy (test set) | 0.9197                                              | 0.9349                                              |
| Model Complexity    | Low                                                 | High                                                |
| Training Time       | <i>Not measured by notebook (theoretically Low)</i> | <i>Not measured by notebook (theoretically Low)</i> |
| Interpretability    | High                                                | Low                                                 |

Model Complexity, Training Time (qualitative) and Interpretability rows follow standard theoretical characterizations of these algorithm families as stated in the experiment sheet; the notebook did not measure wall-clock training time for either model.

**Observations:**

- **Best-performing classifier:** SVM (RBF kernel) achieved the highest test accuracy (0.9349), the highest tuned cross-validation accuracy (0.9313), and the highest 5-fold CV average accuracy (0.9231) among all models evaluated, making it the best-performing classifier in this experiment.
- **Impact of regularization:** Tuning Logistic Regression's  $C$  and solver via GridSearchCV improved its cross-validated accuracy from the baseline model's test accuracy of 0.9197 to a best CV accuracy of 0.9258, showing that selecting an appropriate regularization strength ( $C = 10$ , i.e. relatively weak regularization) improved generalization on this dataset.
- **Kernel behavior in SVM:** The RBF kernel outperformed Linear and Sigmoid kernels, and substantially outperformed the Polynomial kernel (0.9349 vs. 0.7644 accuracy), indicating the decision boundary between spam and ham in this feature space is better captured by a smooth non-linear (RBF) boundary than by a fixed-degree polynomial boundary.
- **Bias–variance trade-off:** Both models show small train-test accuracy gaps (Table 6), indicating neither is strongly overfitting (low variance); Logistic Regression's slightly lower accuracy across the board suggests marginally higher bias (a linear decision boundary) compared to the RBF-kernel SVM, which can model non-linear boundaries and achieves both higher training and test accuracy.

## 13 Learning Outcomes

- Understood the working of probabilistic (Logistic Regression) and margin-based (SVM) classifiers.
- Applied hyperparameter tuning using GridSearchCV with 5-fold cross-validation for both models.
- Evaluated classification models using Accuracy, Precision, Recall, F1-Score, Confusion Matrix and ROC Curve.
- Interpreted experimental results to compare model complexity, interpretability, and generalization behavior between linear and kernel-based classifiers.

## 14 Conclusion

In this experiment, Logistic Regression and Support Vector Machine classifiers were implemented to classify emails from the Spambase dataset as spam or ham. After standardizing the 57 numerical features and splitting the data 80:20, both models were trained, evaluated, and tuned using `GridSearchCV` with 5-fold cross-validation. The tuned Logistic Regression model ( $C = 10$ , solver=liblinear) achieved a best CV accuracy of 0.9258, while the tuned SVM (RBF kernel,  $C = 1$ , gamma=scale) achieved a best CV accuracy of 0.9313 and the highest overall test accuracy (0.9349). Among the four SVM kernels compared, RBF performed best and Polynomial performed worst. Both models showed only a small gap between training and test accuracy, indicating good generalization without strong overfitting. Overall, SVM with the RBF kernel was the best-performing classifier in this experiment, at the cost of lower interpretability and higher model complexity

compared to Logistic Regression.

## 15 References

1. Scikit-learn Documentation, *Logistic Regression*. Available: [https://scikit-learn.org/stable/modules/linear\\_model.html#logistic-regression](https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression)
2. Scikit-learn Documentation, *Support Vector Machines*. Available: <https://scikit-learn.org/stable/modules/svm.html>
3. Scikit-learn Documentation, *Hyperparameter Optimization*. Available: [https://scikit-learn.org/stable/modules/grid\\_search.html](https://scikit-learn.org/stable/modules/grid_search.html)
4. Kaggle, *Spambase Dataset*. Available: <https://www.kaggle.com/datasets/somesh24/spambase>
5. UCI Machine Learning Repository, *Spambase Data Set*. Available: <https://archive.ics.uci.edu/dataset/94/spambase>